Computer Organization and Architecture (CSL251)
LAB-2
2025-26 W
RISC-V Assembly Debugging Using Spike

Program:

nano ins_1.s

```
  GNU nano 7.2
.data                   # Data section: used to store variables in memory

value:
        .byte 0x10       # Store one byte with value 0x10 (decimal 16)

value1:
        .byte 0xF6       # Store one byte with value 0xF6
                         # As a signed 8-bit value, this represents -10

.text                   # Code section (instructions)
.global main             # Make 'main' visible to the linker

main:
        lb t0, value     # Load 1 byte from memory at 'value'
                          # 'lb' = load byte (signed)
                          # 0x10 → sign-extended → t0 = 16

        lb t1, value1    # Load 1 byte from memory at 'value1'
                          # 0xF6 has MSB = 1, so it is negative
                          # Sign-extended → t1 = -10 (0xfffffffffffffff6)

        add t0, t0, t1   # Add contents of t0 and t1
                          # t0 = 16 + (-10) = 6
```

Compile the Program(produces the executable file a.out):

riscv64-unknown-elf-gcc ins_1.s

Disassemble the Executable:

riscv64-unknown-elf-objdump -D a.out > ins_1_dump

```
:~/COA-programs/INSTRUCTIONS$ riscv64-unknown-elf-gcc ins_1.s
:~/COA-programs/INSTRUCTIONS$ riscv64-unknown-elf-objdump -D a.out >  ins_1_dump
```

Locate the <main> label and Copy the address of the first instruction:

vi ins_1_dump

```
~/COA-programs/INSTRUCTIONS$ vi ins_1_dump
```

```
   101e6:       00000067                jr      zero # 0 <exit-0x10120>
   101ea:       8082                    ret

00000000000101ec <main>:
   101ec:       00002297                auipc   t0,0x2
   101f0:       e2c28283                lb      t0,-468(t0) # 12018 <__DATA_BEGIN__>
   101f4:       00002317                auipc   t1,0x2
   101f8:       e2530303                lb      t1,-475(t1) # 12019 <value1>
   101fc:       929a                    add     t0,t0,t1

00000000000101fe <__fp_lock>:
   101fe:       4501                    li      a0,0
   10200:       8082                    ret

0000000000010202 <stdio_exit_handler>:
   10202:       6649                    lui     a2,0x12
   10204:       65c5                    lui     a1,0x11
   10206:       6549                    lui     a0,0x12
   10208:       02060613                addi    a2,a2,32 # 12020 <__sglue>
   1020c:       29a58593                addi    a1,a1,666 # 1129a <_fclose_r>
   10210:       03850513                addi    a0,a0,56 # 12038 <_impure_data>
   10214:       aca1                    j       1046c <_fwalk_sglue>

0000000000010216 <cleanup_stdio>:
   10216:       650c                    ld      a1,8(a0)
   10218:       1141                    addi    sp,sp,-16
```

Exit vi editor

Start Spike Debugger:

spike -d pk a.out

```
deepesh@deepesh-HP-Laptop-15-da0xxx:~/COA-programs/INSTRUCTIONS$ spike -d pk a.out
(spike)
```

Initial register dump (before executing code):

(spike) reg 0

```
(spike) reg 0
zero: 0x0000000000000000  ra: 0x0000000000000000  sp: 0x0000000000000000  gp: 0x0000000000000000
  tp: 0x0000000000000000  t0: 0x0000000000000000  t1: 0x0000000000000000  t2: 0x0000000000000000
  s0: 0x0000000000000000  s1: 0x0000000000000000  a0: 0x0000000000000000  a1: 0x0000000000000000
  a2: 0x0000000000000000  a3: 0x0000000000000000  a4: 0x0000000000000000  a5: 0x0000000000000000
  a6: 0x0000000000000000  a7: 0x0000000000000000  s2: 0x0000000000000000  s3: 0x0000000000000000
  s4: 0x0000000000000000  s5: 0x0000000000000000  s6: 0x0000000000000000  s7: 0x0000000000000000
  s8: 0x0000000000000000  s9: 0x0000000000000000 s10: 0x0000000000000000 s11: 0x0000000000000000
  t3: 0x0000000000000000  t4: 0x0000000000000000  t5: 0x0000000000000000  t6: 0x0000000000000000
```

Run Until main (Execute instructions until PC reaches 0x101ec, stop just before executing it)

(spike) until pc 0 101ec

```
(spike) until pc 0 101ec
(spike)
```

Instruction-by-Instruction Execution

Instruction 1 (just press enter that shows):

```
(spike)
core   0: 0x00000000000101ec (0x00002297) auipc   t0, 0x2
```

auipc t0, 0x2 # Compute PC-relative base address for 'value'

t0 = PC + (0x2 << 12)

Used to access global variables

Instruction 2 (just press enter that shows):

```
(spike)
core   0: 0x00000000000101f0 (0xe2c28283) lb      t0, -468(t0)
```

Load signed byte from memory → value (0x10)

.byte 0x10 → decimal 16

Sign-extended to 64-bit

t0 = 16

Register State After Load :

```
(spike) reg 0 t0
0x0000000000000010
```

Instruction 3 (just press enter that shows):

```
(spike)
core   0: 0x00000000000101f4 (0x00002317) auipc   t1, 0x2
```

Compute PC-relative base address for 'value1'

Instruction 4 (just press enter that shows):

```
(spike)
core   0: 0x00000000000101f8 (0xe2530303) lb      t1, -475(t1)
```

Load signed byte from memory → value1 (0xF6)

0xF6 → binary 11110110

MSB = 1 → negative

Signed value = −10

✔ t1 = 0xfffffffffffffff6

Register State After Load :

```
(spike) reg 0 t1
0xfffffffffffffff6
```

Addition Instruction (just press enter that shows):

```
(spike)
core   0: 0x00000000000101fc (0x0000929a) c.add   t0, t1
```

Verify Final Result:

```
(spike) reg 0 t0
0x0000000000000006
```

Final result stored in t0 = 6